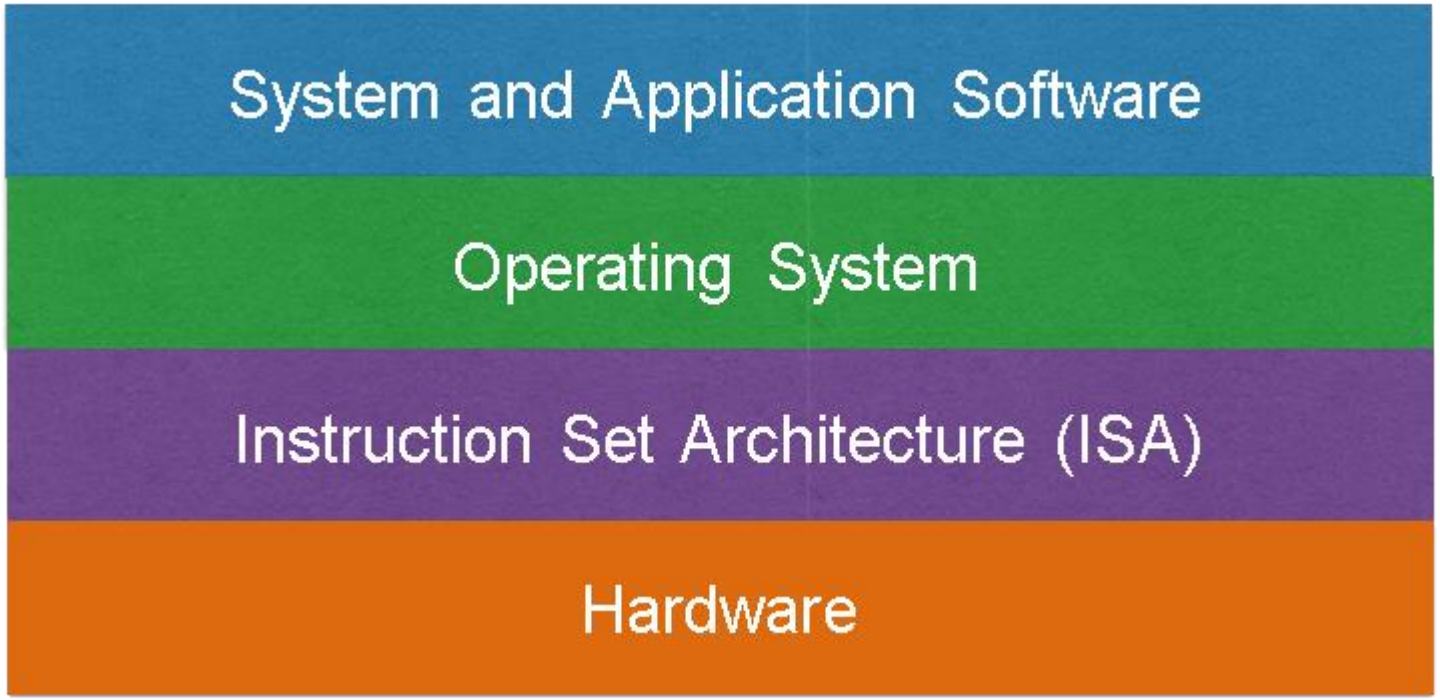
Computer Architecture and Operating Systems
COSC 22073/BECS 22233

Lesson 04: Instruction Set Architecture (ISA)

Department of Statistics & Computer Science
Faculty of Science
University of Kelaniya

What is Instruction Set Architecture (ISA)?

- Serves as an interface between software and hardware. It provides mechanism by which the software tells the hardware what should be done.



Instruction Set Design Issues

- Where are operands stored?
- How many explicit operands are there?
- How is the operand location specified?
- What type & size of operands are supported?
- What operations are supported?

Types of ISAs

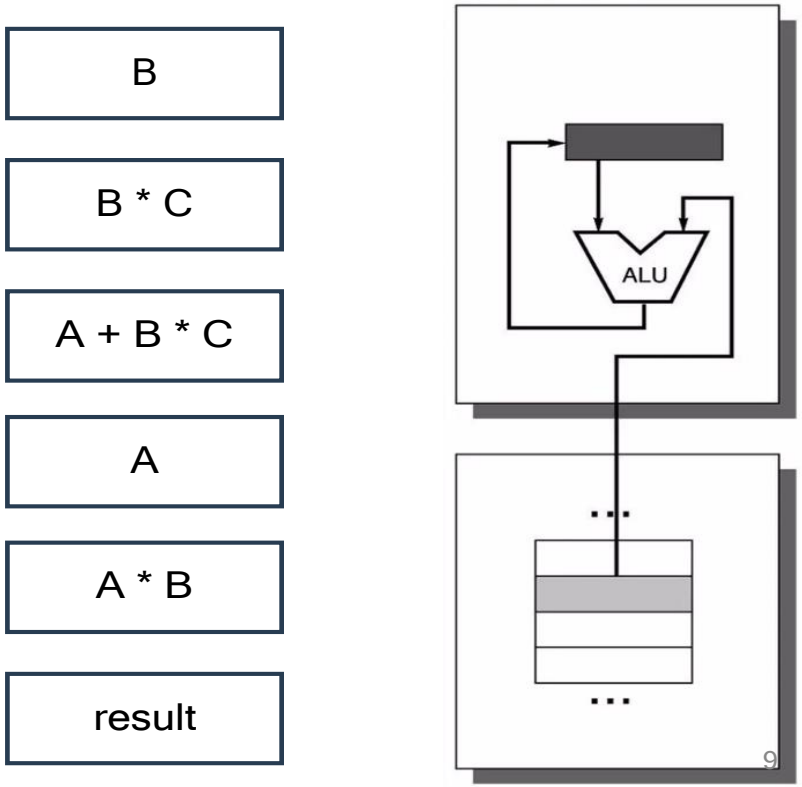
- **Accumulator-Based ISA:**
  - **Characteristics:** In an accumulator-based ISA, one register (the accumulator) serves as the primary location for performing arithmetic and logical operations. Data is typically loaded from memory into the accumulator, processed, and then stored back.
  - **Example:** The early Intel 8008 microprocessor used an accumulator-based architecture.

Types of ISAs

Accumulator-Based ISA:

Example: A * B - (A + C * B)

load B
mul C
add A
store D
load A
mul B
sub D



Types of ISAs

Accumulator-Based ISA:

Pros:

Very low hardware requirements

East to design and understand

Cons:

Accumulator becomes the bottleneck

Little ability for parallelism or pipelining

High memory traffic

Types of ISAs

Stack-Based ISA:

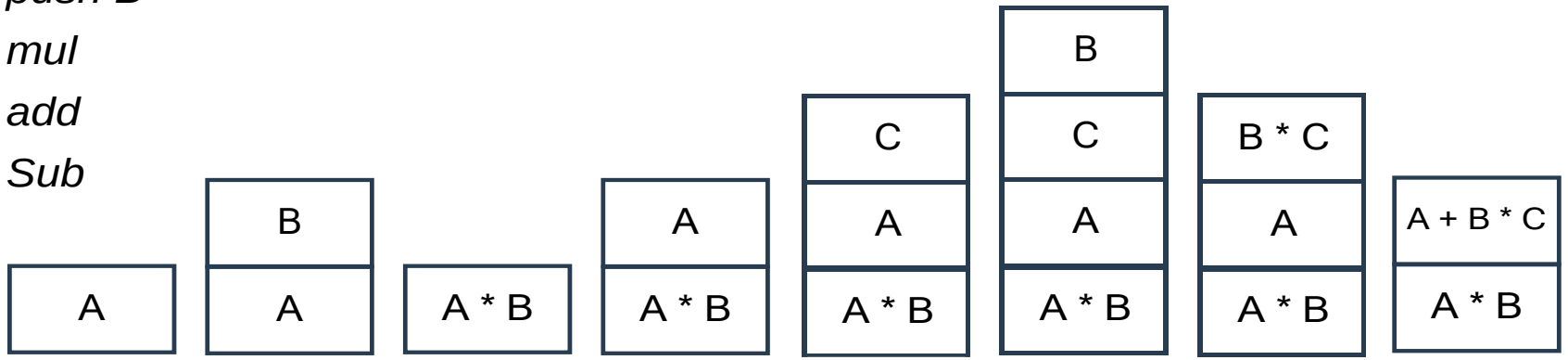
- **Characteristics:** Stack-based ISAs use a stack data structure to store and manipulate data and instructions. Operations often involve pushing data onto the stack or popping data off the stack. Stack-based architectures are commonly associated with some high-level programming languages, such as Forth.
- **Example:** The Java Virtual Machine (JVM) uses a stack-based architecture for executing bytecode.

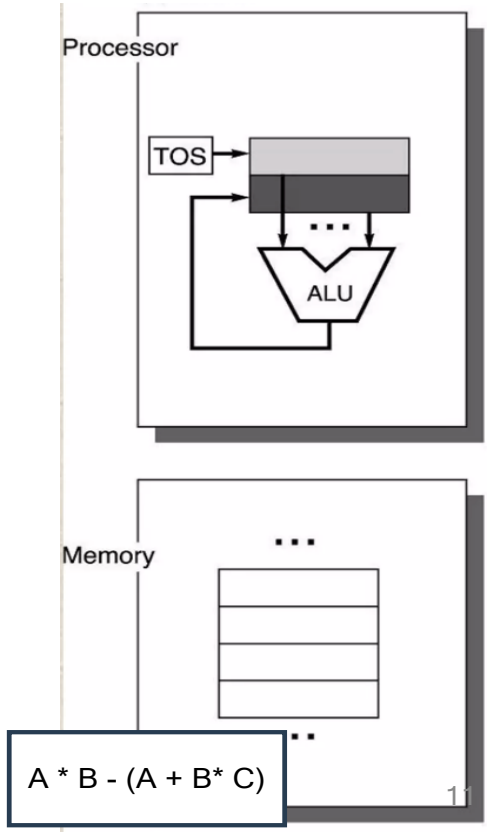
Types of ISAs

Stack-Based ISA:

Example: A * B - (A + C * B)

push A
push B
mul
push A
push C
push B
mul
add
Sub





Types of ISAs

Stack-Based ISA:

Pros:

Low hardware requirements

Good code density

Easy to write a simpler compiler for stack architecture

Cons:

Stack becomes the bottleneck

Little ability for parallelism or pipelining

Data is not always at the top of stack when need, so additional

Types of ISAs

Register-Memory ISA:

- **Characteristics:** In register-memory ISAs, instructions can operate on both registers and memory locations. Data can be loaded from memory into registers or stored from registers into memory.
- **Example**: The x86 ISA is an example of a register-memory architecture, where instructions can access both registers and memory directly.

Types of ISAs

- **Register-Memory ISA:**

Example: A * B - (A + C * B)

load R1, C

mul R1, B

add R1, A

store R1, D

load R2, A

mul R2, B

sub R2, D

Types of ISAs

Register-Memory ISA:

Pros:

Some data can be accessed without loading first

Instruction format easy to encode

Good code density

Cons:

Operands are not equivalent

Variable number of clocks per instruction

May limit the number of registers

Types of ISAs

Register-Register ISA (Load/Store ISA):

Characteristics: In register-register ISAs, instructions typically perform operations only on registers. Memory operations involve loading data from memory into registers or storing data from registers into memory. These ISAs often have separate load and store instructions.

Example: The MIPS ISA is a classic example of a register-register architecture.

A*B - (A+C*B)

3 OPERANDS

```
LOAD R1 A
LOAD R2 B
LOAD R3 C
MUL R4 R1 R2
MUL R5 R3 R2
ADD R6 R1 R5
SUB R7 R4 R6
```

2 OPERANDS

?

Types of ISAs

- **Memory-Memory ISA:**
- **Characteristics:** Memory-memory ISAs are less common and involve instructions that perform operations directly between memory locations without the use of registers. These ISAs can be challenging to implement efficiently due to the need for frequent memory accesses.
- **Example:** The Burroughs B5000 series computers used a memory-memory architecture.

Types of ISAs

- **Memory-Memory ISA:**

Example: A * B - (A + C * B)

3 operands

mul D, A, B

mul E, C, B

add E, A, E

sub E, D, E

2 operands

mov D, A

mul D, B

mov E, C

mul E, B

add E,A

sub E, D

Types of ISAs

- **Memory-Memory ISA:**

Pros:

Requires fewer instructions (especially if 3 operands)

Easy to write compilers

Cons:

Very high memory traffic

Variable number of clocks per instruction

Classification of Instructions

- 4-address instructions
- 3-address instructions
- 2-address instructions
- 1-address instructions
- 0-address instructions

Classification of Instructions

4-address Instructions

The 4-address instruction specifies the two source operands, the destination operand and the address of next instruction

Opcode	Destination	Source 01	Source 02	Next Address

Classification of Instructions

3-address Instructions

The 3-address instruction specifies addresses for both operands as well as result

Opcode	Destination	Source 01	Source 02

Classification of Instructions

- **2-address Instructions**

One field serves two purposes

Opcode	Destination Source 01	Source 02

Classification of Instructions

1-address Instructions

The 1-address instruction has a dedicated CPU register called the accumulator, to hold one operand & the result - No address is needed to specify the accumulator

Opcode	Destination Source 01

Classification of Instructions

0-address Instructions

The 0-address instruction uses a stack to hold both operands and the result. Operations are performed between the value on the top of the (TOS) and the second value on the stack (SOS) and the result is stored on the TOS

Opcode

Addressing Modes

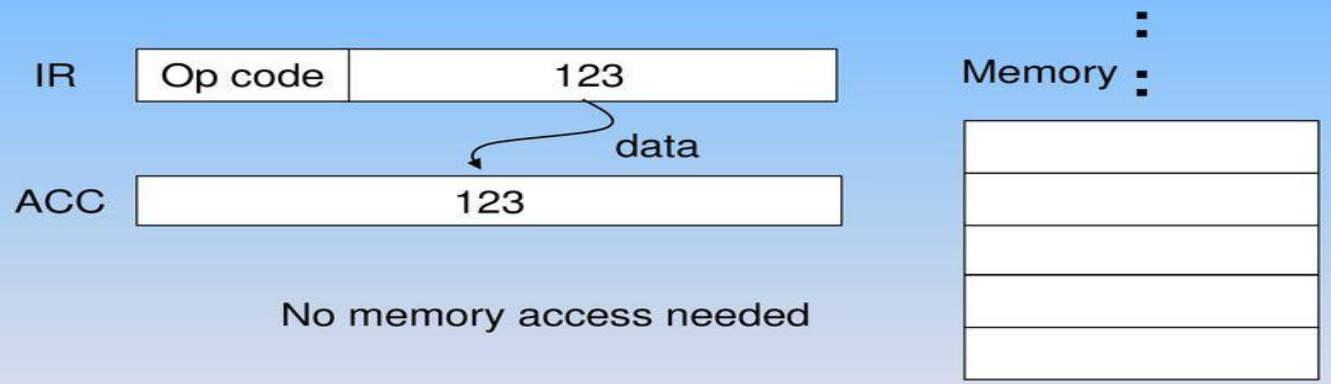
Immediate Addressing Mode

- Data for the instruction is part of the instruction itself.
- No need to calculate any address
- Limited range of operands

Addressing Modes

Immediate addressing mode

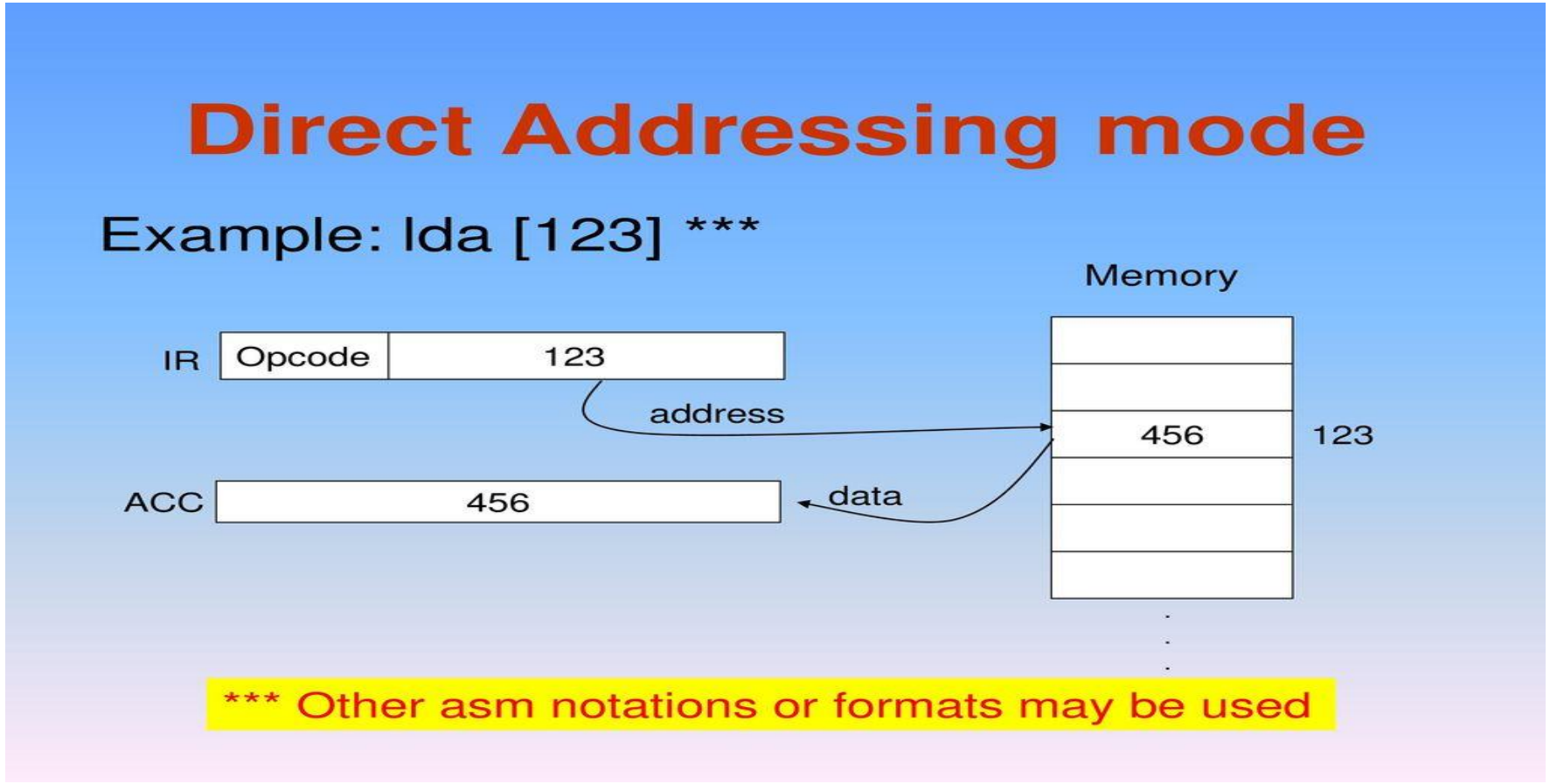
Example: lda 123 ***



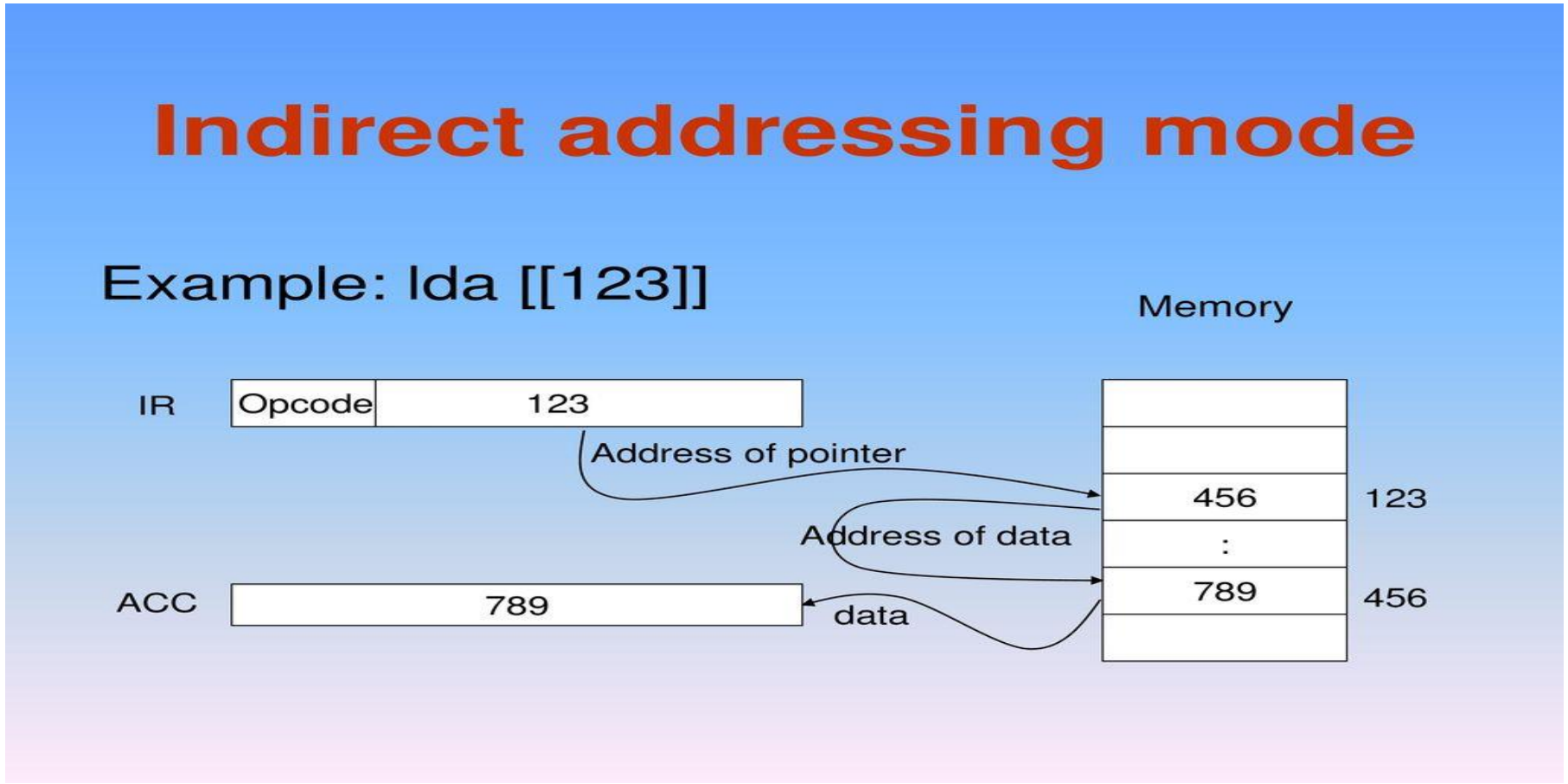
No memory access needed

*** Note we have used the lda instruction as an example to maintain consistency and to limit the number of operands to one only

Addressing Modes



Addressing Modes



RISC (Reduced Instruction Set Computers)

- A concept or philosophy of machine design: not a set of architectural features.
- Underlying idea is to reduce the number and complexity of instructions.

Features of RISC Machines:

- One instruction per clock period
- All instructions have the same size
- CPU accesses memory only for load and store operations
- Simple and few addressing modes

CISC (Complex Instruction Set Computers)

- More work per instruction
- Wide variety of addressing modes
- Variable instruction lengths and execution times per instruction
- CISC machines attempt to reduce the “semantic gap”

Disadvantages of CISC:

- Clock period, T, cannot be reduced beyond a certain limit.
- Complex addressing modes delay operand fetch from memory
- Difficult to make efficient use of speedup techniques

Thank You.